MSCS & MSDS OOP with Python Assignment 2
Advent 2026

Instructions

- Submit **one GitHub repository link** on Moodle. Use one Jupyter Notebook per mini-project (e.g. `project1_population.ipynb`). Put reusable logic in `.py` modules under `src/` and import it into the notebooks.
- The repository must include a `README.md` (how to set it up and run it, plus a summary of findings), a `requirements.txt` or `environment.yml`, and a `tests/` folder.
- **For mini-project 2, OOP is required.** The project must model its domain with at least one well-designed class: attributes, methods, docstrings and type hints. Use inheritance, composition or dunder methods where they genuinely help.
- **Reproducibility:** fix random seeds (`np.random.default_rng(seed)`). A notebook must run top to bottom with *Restart & Run All* without errors.

> **Academic integrity & AI use:** You may use AI coding assistants, but you must declare how you used them in your README. You may be asked to explain any line of your code in a short viva. Identical repositories will be investigated.

Mini-Project 1: Fetching and Displaying a Weather Forecast with Python Web APIs

1. Overview

You will write a Python program that takes a location name, finds its coordinates, requests a weather forecast from Opensource APIs, and displays the forecast clearly. You will use Python's standard libraries and the widely used requests module.

2. Approved APIs

Provider	Purpose
Open-Meteo Geocoding (geocoding-api.open-meteo.com/v1/search)	Location name to latitude and longitude
Open-Meteo Forecast (api.open-meteo.com/v1/forecast)	Main forecast
MET Norway Locationforecast 2.0 (api.met.no/weatherapi/locationforecast/2.0)	Second source (Part C)

Read the API documentation before coding. Use your own university email in the User-Agent, and never invent contact details. Keep traffic low, and cite the data source in your output.

3. Tasks

Part A: HTTP fundamentals

1. Using only urllib.request, urllib.parse and json, geocode a city name such as "Dresden". Build the query string with urllib.parse.urlencode.
2. Repeat the same call with requests, using the params= argument and a timeout.
3. In a Markdown cell, compare the two approaches (code length, error handling, header and encoding control).
4. Inspect and report the status code, the Content-Type header, and the response time for one request.

Part B: Forecast retrieval and display

1. Write a function geocode(name). It returns candidate locations. Handle zero results, because the results key may be absent. Handle ambiguous names (for example "Springfield") by listing the candidates and letting the user pick.
2. Write a function get_forecast(lat, lon, days). Request at least daily maximum and minimum temperature, precipitation sum, and maximum wind speed. Set timezone so the dates match the location.
3. Convert the JSON into a pandas DataFrame with clear column names and units. Decode the weather code into a readable description using the mapping in the Open-Meteo documentation.
4. Display the result below with a text summary
    - a formatted table;
    - a chart of the temperature range with precipitation

Part C: Robustness and second source

1. Wrap every request in a function that:
    - sets a timeout;
    - calls raise_for_status();
    - retries on failures such as connection errors and 5xx responses, with exponential backoff and a retry cap;

    - raises a clear custom exception on final failure.
2. Show the behaviour with deliberately broken inputs: an invalid city, an invalid latitude, no network (simulated), and a tiny timeout.

Part D: Report

Write 1,000–1,500 words in Markdown cells in the notebook.

- the data flow from user input to displayed forecast, with a simple diagram;
- the licensing and usage terms of each API, and what they mean for a real product;
- the reliability issues you met, and how your code handles them;
- the privacy implications of sending location queries to third parties;

4. Requirements and Constraints

- The notebook must run top to bottom after "Restart & Run All". The only manual input is a configuration cell holding the location and your contact email.
- Do not hardcode contact details or secrets in shared code. Use an environment variable or a config cell with a placeholder.
- Write reusable functions with docstrings and type hints. Keep cells short.
- Allowed: urllib, requests, json, datetime, time, os, pandas, matplotlib. Other libraries need a one-line justification.
- Do not use a wrapper library for the weather APIs. You must make the HTTP calls yourself.
- Include at least five assert-based or pytest checks on your parsing functions. Use saved sample JSON so they run offline.

5. Deliverables

1. One executable .ipynb notebook.
2. Sample JSON files used by your offline checks.

Mini-Project 2: Uganda Tax Engine

1. Context

The Uganda Revenue Authority (URA) administers several taxes with different calculation rules. You will design an extensible, testable tax calculator that models these rules.

2. Sample regulatory baseline

Tax	Baseline rule to implement
PAYE (resident, monthly)	Nil up to 335,000. 20% of the excess over 335,000 up to 410,000. 15,000 + 25% of the excess over 410,000 up to 485,000. 33,750 + 30% of the excess over 485,000 up to 10,000,000. Above 10,000,000, add 10% of the excess over 10,000,000 [ura.go][pulse]
NSSF	Employee 5% of gross, employer 10% [calculator.sunpro]
VAT	Standard rate 18% [leadafrik]
Corporate income tax	30% for resident companies [leadafrik][taxatlas]
Local Service Tax	Optional deduction applied before PAYE [afrotools]

Confirm the tax rates against the URA documentation and cite the version you used.

3. Functional requirements

1. **PAYE calculator.** Take monthly gross pay, allowances, and residency status. Apply the progressive bands and the 10% surcharge above 10,000,000. Resident and non-resident schedules must both be supported.
2. **Payroll pipeline.** Compute NSSF, Local Service Tax (configurable), PAYE, and net pay. Document the order of deductions and justify it.
3. **VAT module.** Support standard-rated, zero-rated, and exempt supplies. Compute output VAT, input VAT, and net VAT payable for a list of invoices.
4. **Corporate income tax module.** Compute chargeable income as revenue minus allowable deductions, then apply the rate.

4. Mandatory design constraints

- **Abstraction.** Define an abstract Tax base class with a calculate(self, ctx: TaxContext) -> TaxResult method.
- **Money handling.** Use decimal.Decimal only, never float. Implement a Money value object in UGX with explicit rounding rules and operator overloading (__add__, __mul__, __lt__, and __repr__).
- **Validation and errors.** Create a custom exception hierarchy (TaxError, InvalidIncomeError, NoScheduleForDateError). Never fail silently.

5. Testing requirements

- Boundary tests at every band edge (335,000; 335,001; 410,000; 485,000; 10,000,000; 10,000,001).
- Property-based tests with hypothesis: tax must be non-negative, monotonic in income, and never exceed income.

6. Extension tasks (choose one)

- **A.** Add withholding tax and rental income tax modules.
- **B.** Add a TaxPlanner that compares two scenarios (for example, salary versus dividend) and reports the difference.

7. Discussion questions for the report

1. Why is float unsafe for tax computation? Show a concrete failing example.

2. Where would inheritance be justified here, and where did you deliberately avoid it?

8. Deliverables

1. One executable .ipynb notebook.

Assessment Rubric (applied to each mini-project)

Criterion	Weight	What an excellent submission shows
Correctness & completeness	45%	All tasks done, results correct.
Software design (OOP)	30%	Clear class responsibilities, no duplicated logic, sensible naming, docstrings, type hints, input validation.
Visualisation	10%	Labelled axes with units, titles, legends.
Testing & reproducibility	10%	Meaningful tests pass; *Restart & Run All* works; clear README.
Extension task	5%	At least one extension attempted with a thoughtful write-up.